Idempotent Wallet: Complete Study Guide (IT3133)

Oct 16, 2026 · @Pehesara

1. The story in one page

The one-sentence version. We built a digital wallet where pressing "Send" five times still moves the money once, and we prove it by breaking the system on purpose in front of the examiners.

What the lecturer is really testing. The module is *Programming Distributed Components*. Every slide leads to one idea: when components talk over a network, things fail halfway, and the same request can arrive twice. Slide 4 of *Distributed Design* gives the cure in one line: **at-least-once delivery + idempotent handlers keyed by a stable operation ID**. Our project is a working, testable proof of that sentence.

The story in five acts

1. **The problem.** A customer taps "Send LKR 500". The phone shows a spinner, the Wi-Fi drops, the customer taps again. Did the money move once or twice? The server cannot tell a lost request from a lost reply, so it cannot know.
2. **The rule.** Never overwrite a balance. Only append a new line to a permanent statement (an *event*). This is event sourcing, and it gives us a full audit trail.
3. **The guard.** Every money-moving request carries an `Idempotency-Key`. The database remembers *key → result*. A repeat gets the stored receipt back instead of a second debit.
4. **The pipeline.** Events leave the database through an *outbox*, travel through RabbitMQ, and consumers skip duplicates using an *inbox*. A transfer between two wallets runs as a *saga* across two services, with a refund if the second step fails.
5. **The proof.** We inject failures on purpose (lost responses, duplicate messages, a crashed service, a flaky payment provider) and show that the one rule always holds: **money is never created or destroyed by a retry**.

Why "Exactly-Once(ish)". True exactly-once delivery over a network is impossible. What we build is *effectively-once*: the message may be delivered many times, but its **effect** happens once. The "ish" is the honest word for that gap, and saying it confidently in the viva is worth marks.

What is in the project

Piece	What it is	Port / place
`wallet-service`	Wallets, the ledger, the idempotent API, outbox relay, consumers, the mock payment provider	8080
`transfer-service`	The saga that moves money between two wallets	8081
PostgreSQL	Two databases: `wallet` and `transfers`	5432
RabbitMQ	The message broker (middleware) between the services	5672, UI on 15672
Tests	Unit tests, end-to-end tests, a k6 load script	`src/test`, `k6/`
Demo files	`requests.http` files and the README	project folders

In numbers: **2 services, 2 databases, 1 broker, 3 idempotency layers, 11 demo scenarios**.

How to read the rest of this guide. Each topic follows the same order: a one-line definition, then an everyday analogy, then how it appears in our code. Where the code has a limit, I say so plainly (section 13), because examiners reward students who name their own weak spots.

2. Foundations: the theory under everything

Every later section is a consequence of five ideas. Learn these first and the rest becomes obvious.

2.1 Partial failure

Definition. A distributed system is several programs on different machines cooperating by sending messages. Its defining problem is *partial failure*: one part can fail while the others keep running.

Analogy. Texting a friend. "Delivered" does not mean "read", and silence does not mean "no". In a single program a crash stops everything, so you always know where you stand. Across a network you often do not.

2.2 The lost-reply problem (the heart of the project)

A client sends "transfer LKR 500" and then hears nothing. There are three possible worlds, and the client cannot tell which one it is in:

1. The request never arrived. Money did not move.
2. The server crashed before saving. Money did not move.

3. The server saved the transfer, but the **reply was lost**. Money moved.

The only safe move for the client is to **retry**. So retries are not optional, and therefore **duplicates are unavoidable**. The whole project exists to make a duplicate harmless. (This is a cousin of the classic *Two Generals problem*: two parties over an unreliable link can never be fully certain the other received the last message.)

2.3 Delivery guarantees

Guarantee	Meaning	How it works	Weakness
At-most-once	Delivered 0 or 1 times	Send once, never retry	Messages can be lost
At-least-once	Delivered 1 or more times	Retry until acknowledged	Messages can be duplicated
Exactly-once	Delivered exactly 1 time	Cannot be guaranteed over a network	Impossible in general

Our recipe: use at-least-once (so nothing is lost), then make the receiver **idempotent** (so duplicates change nothing). The result is *effectively-once*. RabbitMQ is at-least-once, HTTP clients retry, and our handlers absorb the duplicates.

2.4 Idempotency

Definition. An operation is idempotent if doing it many times has the same effect as doing it once. In maths: $f(f(x)) = f(x)$.

Analogy. A lift button: pressing it five times still calls one lift. Compare a light switch labelled "set to ON" (idempotent) with one labelled "toggle" (not idempotent).

Operation	Idempotent?	Why
"Set balance to 500"	Yes	Same end state however often you run it
"Add 500 to balance"	**No**	Every run changes the result
HTTP GET, PUT, DELETE	Yes (by specification)	They describe a state, not an increment
HTTP POST (create a transfer)	**No** by default	Each call creates something new

Money movements are "add 500" operations, and they arrive through POST, so they are naturally **not** idempotent. We have to *make* them idempotent. The technique: give every **intention** a stable identity (the `Idempotency-Key`) and store *key → result*. The identity belongs to the intention ("this one transfer"), not to a single network attempt, so every retry carries the same key.

2.5 Consistency, CAP and eventual consistency

CAP theorem (lecture 03). During a network partition you choose between Consistency (everyone sees the latest write) and Availability (everyone gets an answer). Partition tolerance is not optional.

Our position. Ledger writes favour **consistency per wallet**: a conflicting write is rejected and retried rather than silently accepted. Where messages cross services (notifications, saga replies) the system is **eventually consistent**: the other side catches up a moment later.

2.6 Ordering without clocks

The lectures cover Lamport and vector clocks for ordering events across machines. We do not need them here, because each wallet is owned by one service and gets its own counter, `seq` (1, 2, 3...). One total order *per wallet* is enough. Choosing the simplest tool that works is itself a design skill worth mentioning in the viva.

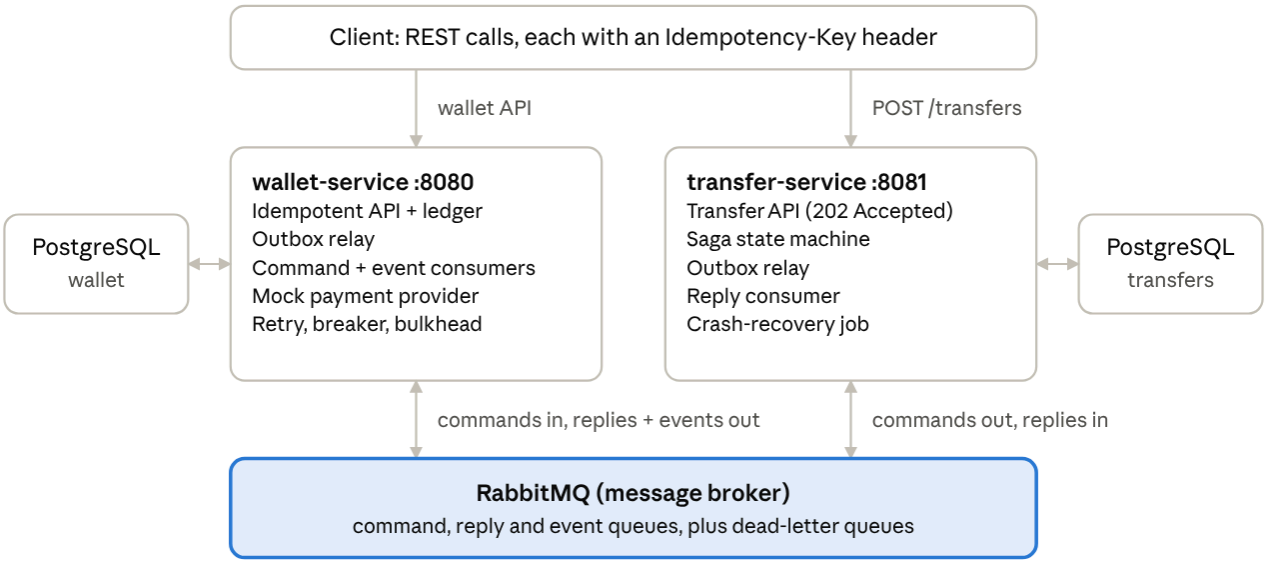
2.7 Where each idea comes from in the slides

Idea	Lecture	Used in the project as
Partial failure, retries, backoff	01 Introduction, slide 24	Retry with jitter, timeouts
CAP, eventual consistency	03 Distributed Programming	Consistency choices above
Idempotency and Exactly-Once(ish)	04 Distributed Design, slides 4 to 5	The three idempotency layers
Outbox, Saga, Resilience, CQRS	04 Distributed Design, slides 3 to 7	Sections 5 to 9 below
Message-oriented middleware	05 Middleware	RabbitMQ
Race conditions, deadlocks	06 Parallel Processing	Optimistic concurrency, parallel tests

Idea	Lecture	Used in the project as
Stubs, mocks, boundaries	07 Component Testing	Tests (section 10)

3. Architecture

Two services share nothing except messages



Each service owns its database. Neither reads the other database.

architecture - 2 services, 1 broker, 2 databases

Clients call either REST API. The two services never call each other directly and never read each other's tables. They only exchange commands and replies through RabbitMQ.

Why two services? The module is about *distributed components*. Two services with separate databases is the smallest design that gives us a real network boundary, so a real saga, real duplicate messages and real partial failures.

Design rules behind the picture

- **Database per service.** Each service owns its data. No shared tables means no hidden coupling.
- **Talk by messages for anything multi-step.** The broker decouples the services in time: if one is down, messages wait.
- **Idempotent at every boundary.** Every place a message or request can repeat has a duplicate guard.
- **One writer per wallet.** Only wallet-service writes a wallet's ledger, so ordering is simple.

Flow A: a top-up (one service)

1. Client sends `POST /wallets/{id}/topups` with an `Idempotency-Key`.
2. `TraceFilter` gives the request a trace ID.
3. `IdempotentExecutor` opens a database transaction and claims the key.
4. `LedgerService` appends a `Credited` event, updates the balance row and writes an outbox row, all in that same transaction.
5. The transaction commits and the receipt is stored against the key. The client gets `201`.
6. `OutboxRelay` (runs every second) publishes the outbox row to RabbitMQ with a stable message ID.
7. `NotificationConsumer` records the message ID in the inbox and writes a notification in one transaction. A repeated message is skipped.

Flow B: a transfer (two services)

1. Client sends `POST /transfers` to transfer-service. It saves the transfer as `PENDING` and a `DEBIT` command in its outbox, then answers `202 Accepted`.
2. The relay publishes the command. RabbitMQ routes it to the wallet service's command queue.
3. `CommandConsumer` debits the source wallet, using the key `transferId:DEBIT`, and writes a reply to its own outbox.
4. The wallet relay publishes the reply. transfer-service's `ReplyConsumer` receives it.
5. The saga moves `PENDING → DEBITED` and queues `CREDIT`. A success reply ends in `COMPLETED`; a failed credit triggers `REFUND` and ends in `COMPENSATED`.

Who does what

Component	Responsibility	Idea it demonstrates
`WalletController`, `CardTopUpController`	Public REST API for wallets, top-ups, withdrawals, card top-ups	Idempotent API design
`IdempotentExecutor`	Claims the key, runs the effect, stores the receipt, in one transaction	Idempotency layer 1
`LedgerService`	Appends events, updates the balance, writes the outbox row, replays history	Event sourcing, optimistic concurrency

Component	Responsibility	Idea it demonstrates
`OutboxRelay`	Publishes outbox rows to RabbitMQ	Transactional outbox
`NotificationConsumer`	Applies an event once using the inbox	Idempotency layer 2
`CommandConsumer`	Executes saga commands once per `transferId:step`	Idempotent commands
`ProviderClient`, `MockProviderController`	Calls an outside payment provider safely	Idempotency layer 3, resilience
`SagaService`, `TransferController`, `ReplyConsumer`	Runs the transfer state machine with refunds and recovery	Saga, compensation
`ChaosController`, `ChaosState`	Switches that break things on purpose for the demo	Fault injection
`TraceFilter`	One trace ID per request	Observability
Flyway `V1__init.sql`	Creates every table, versioned	Database migration
`docker-compose.yml`	Starts PostgreSQL and RabbitMQ with one command	Reproducible environment

4. Data layer: PostgreSQL and the tables

The database is not just storage here. It is the **referee** that makes the guarantees possible, because it can say "no" atomically even when many requests collide.

4.1 Transactions and ACID

Definition. A transaction is a group of changes that succeed or fail as one unit. ACID describes its four promises.

Letter	Promise	Analogy	Where we rely on it
Atomicity	All changes happen, or none	A bank transfer: debit and credit both, or neither	Key row + ledger event + outbox row commit together

Letter	Promise	Analogy	Where we rely on it
Consistency	Rules (constraints) are never broken	A form that refuses a blank required field	`amount_minor > 0`, unique `(wallet_id, seq)`
Isolation	Concurrent transactions do not see each other's half-done work	Two cashiers using separate notepads until each is done	Concurrent duplicates are serialised by the unique index
Durability	Once committed, it survives a crash	Ink, not pencil	A committed event is never lost

Isolation level. PostgreSQL's default is *READ COMMITTED*: each statement sees only data committed before it began. Our code is written for this. When two identical requests insert the same key, the second `INSERT` **waits** for the first transaction to finish. If the first commits, the second sees the row and replays it. If the first rolls back, the second becomes the owner.

4.2 Constraints: rules the database enforces

- **PRIMARY KEY / UNIQUE:** no two rows may share the value. This is our *doorman*.
- **CHECK:** a rule on values, for example `amount_minor > 0`.
- **FOREIGN KEY:** a row must point at a real parent (an event needs a real wallet).
- **`ON CONFLICT DO NOTHING`:** "try to insert; if the key exists, quietly do nothing and tell me zero rows changed". It is how we ask the doorman safely.

4.3 Why money is a `bigint`, never a decimal

We store **minor units** (LKR 10.00 is `1000`). Floating-point numbers cannot represent values like 0.1 exactly, so sums drift (0.1 + 0.2 gives 0.30000000000000004). Whole numbers never drift.

4.4 Every table and why it exists

Table	Database	Purpose	The detail that matters
`wallets`	wallet	One row per wallet	Parent for events and balance
`ledger_events`	wallet	**Source of truth**: every credit and debit, append-only	`UNIQUE (wallet_id, seq)` plus a trigger that forbids update and delete

Table	Database	Purpose	The detail that matters
`wallet_balance`	wallet	Read model: current balance and last `seq`	Updated only `WHERE last_seq = ?` (optimistic concurrency)
`idempotency_keys`	both	Layer 1: key, request hash, stored response, expiry	Primary key `(scope, idem_key)`
`outbox`	both	Messages waiting to be published	Same transaction as the data; partial index on `PENDING` rows
`inbox`	wallet	Layer 2: message IDs already processed	Primary key `(consumer, message_id)`
`notification_log`	wallet	Proof of the consumer's side effect	`UNIQUE (message_id)`: one row even if delivered twice
`transfers`	transfers	Saga state, one row per transfer	`state` column is the state machine

4.5 Three SQL ideas worth knowing by heart

1. Append-only ledger, enforced by the database itself. Even a buggy program cannot rewrite history:

```
CREATE TRIGGER ledger_events_immutable
    BEFORE UPDATE OR DELETE ON ledger_events
    FOR EACH ROW EXECUTE FUNCTION forbid_ledger_change();
```

2. The doorman. The unique index plus `ON CONFLICT DO NOTHING` decides who owns a key:

```
INSERT INTO idempotency_keys (scope, idem_key, request_hash, status,
expires_at)
VALUES (?, ?, ?, 'IN_PROGRESS', now() + interval '24 hours')
ON CONFLICT (scope, idem_key) DO NOTHING
```

3. Several relays without double publishing. `SKIP LOCKED` makes a second worker skip rows another worker already holds:

```
SELECT id, event_id, topic, payload FROM outbox
WHERE status = 'PENDING' ORDER BY id LIMIT 50
FOR UPDATE SKIP LOCKED
```

4.6 Flyway migrations

Definition. Flyway applies numbered SQL scripts (`V1__init.sql` , `V2__...`) in order and records which ran. **Analogy:** version control for the database structure. Rule: never edit a script that has already run; add a new version instead. This keeps every machine in the group identical.

5. Event sourcing and CQRS: the ledger

5.1 Event sourcing

Definition. Instead of storing the *current state* (balance = 600), store the *history of facts* that produced it (credited 1000, debited 400). The current state is calculated by adding the history up.

Analogy. A bank statement versus a whiteboard. A whiteboard shows only today's number, and wiping it destroys the past. A statement is a list of lines that are never erased; the balance is just the running total.

Events are facts in the past tense: `Credited` , `Debited` . They are never edited. A mistake is fixed by appending a new compensating event, exactly like a bank reversal.

Benefit	Cost
Full audit trail: you can say who moved what and when	Reading the balance means adding up events (we avoid this with a stored balance)
Replay: rebuild the state at any time, or at any past moment	The stored balance is a second copy that must stay in step
Natural fit for messaging: each event can be published	Changing an event's shape later needs care
Bugs are recoverable: fix the code, replay the events	More concepts for a team to learn

5.2 CQRS (Command Query Responsibility Segregation)

Definition. Use one model for *changing* data (commands) and a different model for *reading* it (queries).

Analogy. A cashier's ledger book (careful, append-only, authoritative) versus the notice board in the lobby (quick to read, built from the ledger).

In our code.

- **Write side:** `ledger_events`, the source of truth.
- **Read side:** `wallet_balance`, a *projection* built from the events.
- **Honest simplification:** both live in one database and the projection is updated in the **same transaction**, so it is always up to date. Real CQRS usually updates the read side asynchronously (eventual consistency). Say this in the viva.

5.3 How `LedgerService.append` works

1. Read the current balance and `last_seq`.
2. Check the rules. A debit larger than the balance throws `InsufficientFundsException`.
3. Compute `seq = last_seq + 1` and insert the event. The unique `(wallet_id, seq)` stops a second writer from taking the same number.
4. Update the balance row only if `last_seq` is still what we read.
5. Insert the outbox row, so the event is published later.

```
long seq = current.lastSeq() + 1;
jdbc.update("INSERT INTO ledger_events (event_id, wallet_id, seq, type,
amount_minor, trace_id) VALUES (?, ?, ?, ?, ?, ?)",
        eventId, walletId, seq, type, amountMinor, traceId);

int updated = jdbc.update("UPDATE wallet_balance SET balance_minor = ?,
last_seq = ? WHERE wallet_id = ? AND last_seq = ?",
        newBalance, seq, walletId, current.lastSeq());
if (updated == 0) {
    throw new DomainErrors.OptimisticConflictException();
}
```

`Propagation.MANDATORY` on this method means it **refuses to run** unless a transaction is already open. That guarantees nobody can write an event outside the idempotent transaction.

5.4 Optimistic concurrency

Definition. Instead of locking a record so others must wait (*pessimistic*), let everyone proceed and **detect** a clash at the moment of writing (*optimistic*). The loser retries on fresh data.

Analogy. Two people editing a shared document. Pessimistic: only one may open it at a time. Optimistic: both edit, and on saving the second is told "someone changed it, reload and redo".

Worked example (the classic overdraft race). A wallet holds 100. Two withdrawals of 80 arrive at the same instant.

1. Both read balance 100 and `last_seq` 1. Both think 80 is fine.
2. Both try to insert event `seq = 2`. The unique index lets **one** in; the other fails with a duplicate-key error.
3. The loser's transaction rolls back. `ConflictRetry` waits a few random milliseconds and runs it again.
4. On the retry it reads balance 20, so the withdrawal now fails with `INSUFFICIENT_FUNDS`.

Without the unique constraint, both would succeed and the wallet would be overdrawn. This is the same race condition taught in the *Parallel Processing* lecture, solved without locks held across wallets, so there is no circular wait and **no deadlock**.

5.5 Replay: the proof the read model is disposable

`GET /wallets/{id}/balance/replay` recomputes the balance purely from events:

```
SELECT COALESCE(SUM(CASE WHEN type = 'Credited' THEN amount_minor ELSE -
amount_minor END), 0)
FROM ledger_events WHERE wallet_id = ?
```

It returns both numbers and `"consistent": true` when they match. In the demo this shows that the stored balance can always be rebuilt from the history. **Not implemented:** snapshots (saving the balance every N events so replay stays fast on very long histories). Mention it as future work.

6. Idempotency in depth (the marks section)

This is the section the lecturer cares about most. Know it well enough to draw it on a whiteboard.

6.1 Three layers, three different duplicate sources

Layer	Where duplicates come from	Guard	Code
1. Client to API	Users double-click, apps retry after a timeout	`Idempotency-Key` header, `idempotency_keys` table	`IdempotentExecutor`

Layer	Where duplicates come from	Guard	Code
2. Broker to consumer	RabbitMQ delivers at-least-once; the relay can publish twice	`inbox` table keyed by message ID	`NotificationConsumer`, `CommandConsumer`
3. Service to outside provider	Our own retries call the provider again	We send a stable key; the provider returns the same charge	`ProviderClient`, `MockProviderController`

One guard is not enough because each boundary is a different place a message can be repeated.

6.2 Layer 1 step by step: `IdempotentExecutor`

Plain English. Before doing anything, the server writes the client's key into a table. The table's unique index allows only one writer per key. The winner does the work and files the receipt. Everyone else is handed the receipt.

```
@Transactional
public Stored execute(String scope, String key, String requestHash,
Supplier<Stored> effect) {
    int inserted = jdbc.update("INSERT INTO idempotency_keys (...) VALUES
(...) ON CONFLICT (scope, idem_key) DO NOTHING", ...);

    if (inserted == 0) {                    // somebody already owns this
key
        Existing e = loadExisting(scope, key);
        if (!e.requestHash().equals(requestHash)) throw new
KeyReuseException();   // 422
        return new Stored(e.code(), e.body(), true);
// replay
    }
    Stored result = effect.get();           // ledger append + outbox row,
SAME transaction
    jdbc.update("UPDATE idempotency_keys SET status = 'COMPLETED',
response_code = ?, response_body = ? WHERE ...", ...);
    return result;
}
```

Situation	What happens	Client sees
First time	Insert wins, effect runs, receipt stored	`201`, `Idempotent-Replayed: false`
Same key, same payload, later	Insert returns 0 rows, hashes match, stored receipt returned	Same body, `Idempotent-Replayed: true`
Same key, different payload	Hashes differ	`422 IDEMPOTENCY_KEY_REUSED`
Same key at the same instant	Second insert **waits** for the first transaction, then behaves like row 2	`Idempotent-Replayed: true`
First attempt rolled back	Its key row vanished, so the waiting request becomes the owner	Normal success
Key older than 24 hours	Expired row is deleted at the start, request treated as new	New effect
No key at all	Rejected before the executor runs	`400 MISSING_IDEMPOTENCY_KEY`

6.3 Why the key and the effect must commit together

There are two ways to get this wrong, and one transaction closes both:

- **Effect saved, key not saved:** a retry finds no key and repeats the effect. Double debit.
- **Key saved, effect not saved:** a retry finds the key and replays a "success" for money that never moved. Silent loss.

Because the key row, the ledger event and the outbox row are written in one transaction, they all exist or none do.

6.4 The design details examiners probe

- **Key scope.** The key is stored with a scope such as `POST /wallets/{id}/topups`, so the same key on a different endpoint or wallet is a different operation.
- **Request hash.** A SHA-256 of a canonical string (scope and amount). It detects a client reusing a key for a *different* request, which is a bug worth failing loudly on.
- **TTL.** Keys expire after 24 hours and a scheduled job (`IdempotencyCleanup`) deletes old rows, so the table cannot grow forever. After expiry a very late retry is treated as new, which is why the TTL must be longer than any realistic client retry window.

- **Header `Idempotent-Replayed`.** Tells the client (and the examiners) that this answer came from the stored receipt.
- **Rejected requests are not remembered.** If the effect throws (for example insufficient funds), the transaction rolls back and the key row disappears, so a retry is evaluated afresh. That is deliberate for user-facing calls: the balance may have changed. In the saga's `CommandConsumer` the opposite is chosen: a business failure is stored as a `FAILED` result, so every repeat of that command gets the same answer.

6.5 Why the retry loop sits *outside* the transaction

`ConflictRetry` wraps the call to `executor.execute(...)`. A transaction that hit a duplicate-key error is already poisoned and must roll back. So each retry needs a **brand-new transaction**, which only happens if the loop is outside the transactional method. The random delay (*jitter*) stops competing requests retrying in lockstep.

6.6 Walk-through: 50 identical requests at once

1. All 50 try to insert the same key. The unique index lets one through; the other 49 **wait**.
2. The winner appends one event, updates the balance and commits.
3. Each waiting request wakes up, finds the committed row with a matching hash, and returns the stored receipt.
4. Result: **one** ledger event, balance +1000, and 49 responses with `Idempotent-Replayed: true`. The counter `wallet.duplicates.suppressed{layer=api}` reads 49.

This is exactly what the e2e test and the k6 script assert.

6.7 Layers 2 and 3

Layer 2 (inbox). The consumer first inserts the message ID into `inbox` with `ON CONFLICT DO NOTHING`. If zero rows were inserted, it is a duplicate: acknowledge and skip. The inbox row and the side effect share one transaction, for the same reason as in 6.3.

Layer 3 (outside provider). A card top-up first charges the mock provider using the key `walletId:Idempotency-Key`. The provider remembers keys, so a retry returns the same `chargeId` instead of charging again. Then we credit the wallet once through layer 1. Honest limit: a *replayed* card top-up still calls the provider before layer 1 notices the replay (see section 13).

6.8 What idempotency does not do

It cannot make an effect *outside* our database safe unless the receiver also deduplicates. A real SMS or email is at-least-once. We state that openly instead of claiming exactly-once.

7. Messaging: middleware, outbox, inbox, dead-letter queues

7.1 Message-oriented middleware (lecture 05) and RabbitMQ

Definition. Middleware is software that sits between applications so they can talk without knowing each other's details. *Message-oriented middleware* (MOM) does it with messages that wait in queues.

Analogy. A post office. You drop a letter off and leave. The letter waits safely until the recipient collects it, even if the recipient was away when you posted it. That is **decoupling in time**.

RabbitMQ vocabulary

Term	Meaning	Analogy
Producer	Sends messages	The sender
Exchange	Receives messages and routes them by a *routing key*	The sorting office
Queue	Stores messages until consumed	A pigeonhole
Binding	Rule linking an exchange to a queue ("send `wallet.*` here")	A sorting rule
Consumer	Receives and processes messages	The recipient
Acknowledgement (ack)	"I processed it, you may delete it"	A signature on delivery
Durable	Survives a broker restart	Letters kept in a safe

Our topology

Exchange	Routing keys	Queue	Used for
`wallet.exchange`	`wallet.credited`, `wallet.debited`	`wallet.events.notification`	Domain events to the notification consumer

Exchange	Routing keys	Queue	Used for
`wallet.exchange`	`reply.debit`, `reply.credit`, `reply.refund`	`transfer.replies`	Saga replies back to transfer-service
`transfer.exchange`	`cmd.debit`, `cmd.credit`, `cmd.refund`	`wallet.commands`	Saga commands to wallet-service
`wallet.dlx`, `transfer.dlx`	queue-specific	`*.dlq` queues	Messages that could not be processed

7.2 The dual-write problem

Problem. A service must change its database **and** announce it on the broker. Those are two separate systems, so you cannot commit both atomically.

- Save, then publish: crash in between and the world never hears about the change.
- Publish, then save: crash in between and the world hears about a change that never happened.

Analogy. Posting a letter and updating your diary as two separate acts. Whichever you do first, a phone call in the middle can leave them out of step.

7.3 The transactional outbox (lecture 04, slide 3)

Idea. Write the message into an `outbox` table **in the same database transaction** as the data change. Now they are atomic. A separate worker, the *relay*, later reads pending outbox rows and publishes them.

How our `OutboxRelay` works

1. Every second it selects up to 50 `PENDING` rows with `FOR UPDATE SKIP LOCKED`.
2. It publishes each row to RabbitMQ, setting the message ID to the row's `event_id`.
3. It marks the row `PUBLISHED`.
4. If publishing fails, it increments `attempts`. After 10 failed attempts the row becomes `DEAD` instead of blocking the queue.
5. If RabbitMQ is simply unreachable, it stops and leaves rows `PENDING` without counting an attempt, since a broker outage is not the message's fault.

```
@Scheduled(fixedDelay = 1000)
@Transactional
public void publishBatch() {
    List<Row> rows = jdbc.query("SELECT ... FROM outbox WHERE status =
'PENDING' ORDER BY id LIMIT 50 FOR UPDATE SKIP LOCKED", ...);
    for (Row r : rows) {
        publish(r);
        jdbc.update("UPDATE outbox SET status = 'PUBLISHED', published_at =
now() WHERE id = ?", r.id());
    }
}
```

The outbox does not give exactly-once. If the relay crashes *after* publishing but *before* marking the row, it publishes again on restart. That is **at-least-once**, and it is exactly why the receiving side needs an inbox.

7.4 The inbox

The consumer records each message ID before acting on it. A repeated ID means a duplicate, so it is acknowledged and skipped. In our demo you can switch on `duplicatePublish`, which makes the relay publish everything twice, and watch the log say `Duplicate message ... skipped` while `notification_log` still holds one row per message.

7.5 Failure handling and the dead-letter queue (DLQ)

Definition. A dead-letter queue holds messages that could not be processed after several attempts, so one bad message cannot block everything behind it. **Analogy:** the "return to sender" shelf at the post office.

Our configuration (`application.yml` and `RabbitConfig`)

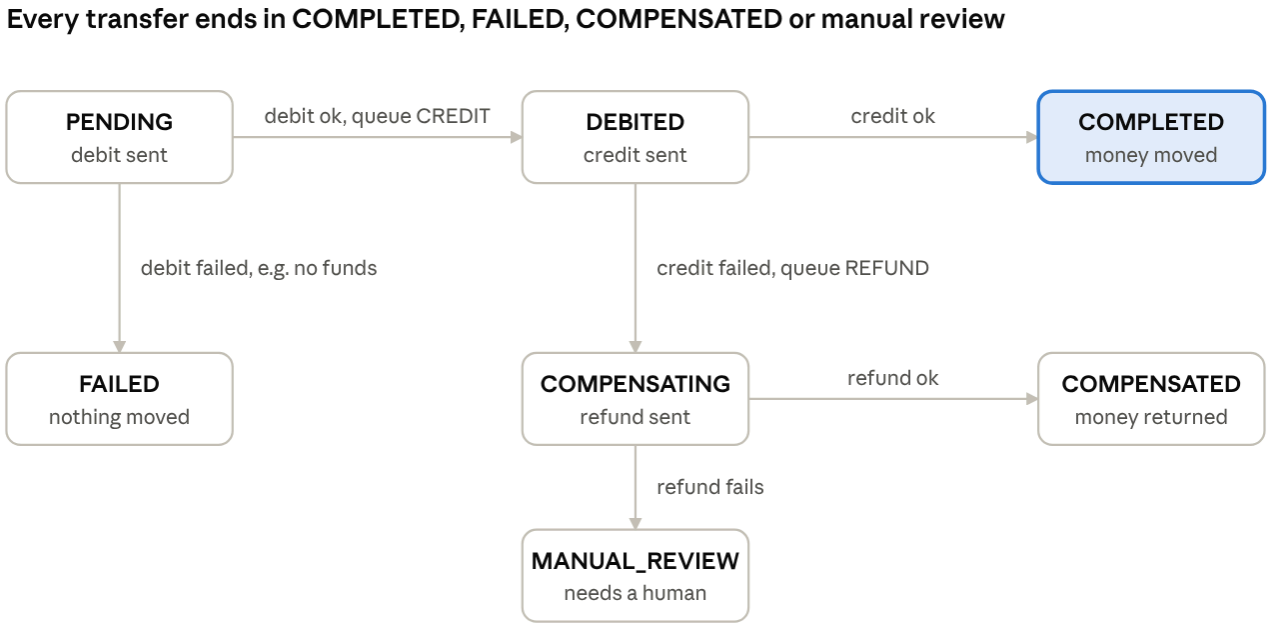
- The listener retries a failing message 3 times with growing delays (0.5 s, then 1 s).
- `default-requeue-rejected: false` means that after the last failure the message is **not** put back.
- The queue is declared with a dead-letter exchange, so the rejected message lands in the `.dlq` queue.
- A *poison message* is one that always fails. The `failConsumer` chaos switch makes every message a poison message so you can show the DLQ filling in the RabbitMQ web page.

Honest limit: nothing replays DLQ messages automatically. A human or a job would inspect them, fix the cause and re-publish. Name this as operational follow-up.

7.6 Ordering

Outbox rows are published in `id` order and carry the aggregate's `seq`. The consumers do not depend on global order. The saga survives reordering because every state change is a guarded `UPDATE ... WHERE state = expected` (section 8).

8. Saga: transfers across two services



Blue is the happy path. Every arrow is a guarded UPDATE ... WHERE state = expected.

transfer saga state machine - 7 states

A transfer is a small state machine stored in the `transfers` table. Each reply from wallet-service moves it one step along an arrow.

8.1 The problem

A transfer debits wallet A (in one database) and credits wallet B (possibly handled elsewhere). A single database transaction cannot span two services. If the debit succeeds and the credit fails, money has vanished.

8.2 Two-phase commit versus a saga

	Two-phase commit (2PC)	Saga
Idea	A coordinator asks everyone to *prepare*, then orders *commit* or *abort*	A chain of local transactions, with a **compensating** action for each step

	Two-phase commit (2PC)	Saga
Atomicity	Strict, all or nothing	Eventual: there are visible in-between states
Locks	Held across services while waiting	None held across services
If the coordinator dies	Participants can block, holding locks	State is in a database; recovery resumes it
Availability	Poor, every participant must be up	Good, messages wait in queues
Lecture view	Rejected for latency, availability and lock contention (slide 12)	Recommended (slides 6 and 11)

Analogy. Booking a holiday: flight, then hotel. If the hotel fails you do not "un-happen" the flight; you perform a new action, *cancel and refund the flight*. That is compensation.

8.3 Orchestration versus choreography

- **Orchestration (our choice):** one coordinator (`SagaService`) tells each participant what to do next. Easy to see, trace and debug. Cost: a central component, which we protect by saving its state in the database.
- **Choreography:** services react to each other's events with no coordinator. Looser coupling, but the overall flow is scattered and harder to follow.

8.4 The transitions

Current state	Reply received	Guarded update	Next command
`PENDING`	`DEBIT` succeeded	`PENDING → DEBITED`	`CREDIT`
`PENDING`	`DEBIT` failed	`PENDING → FAILED`	none
`DEBITED`	`CREDIT` succeeded	`DEBITED → COMPLETED`	none
`DEBITED`	`CREDIT` failed	`DEBITED → COMPENSATING`	`REFUND`
`COMPENSATING`	`REFUND` succeeded	`COMPENSATING → COMPENSATED`	none
`COMPENSATING`	`REFUND` failed	`COMPENSATING → MANUAL_REVIEW`	a human steps in

8.5 Why duplicates and reordering cannot break it

- **Replies are guarded.** A state change is `UPDATE transfers SET state = 'DEBITED' WHERE transfer_id = ? AND state = 'PENDING'`. A duplicate or late reply changes **0 rows** and is ignored, and the next command is queued only when exactly one row changed.
- **Commands are idempotent.** Each command's idempotency key is `transferId:step` (for example `…:DEBIT`). A repeated command is answered from the stored result and never applied twice.
- **The refund is a new fact, not an erase.** It appends a fresh `Credited` event on the source wallet. The ledger therefore shows the debit *and* the refund, which is the audit trail you want.
- **The API is asynchronous.** `POST /transfers` answers `202 Accepted` immediately with a `transferId`; the client polls `GET /transfers/{id}`. The request itself is idempotent through its own `Idempotency-Key`.

8.6 Crash recovery

`SagaService.resendStuck()` runs every 10 seconds. It finds transfers sitting in `PENDING`, `DEBITED` or `COMPENSATING` that have not changed for 30 seconds and re-sends the command for their current step. Because commands are idempotent, re-sending is always safe.

What goes wrong	What saves us
transfer-service dies right after saving `PENDING`	The command is already in its outbox; the relay publishes it after restart
wallet-service is down	The command waits in `wallet.commands`; it is processed on restart
Command lost or never processed	The recovery job re-sends it after 30 s; the wallet dedupes it
A reply is processed twice	The guarded `UPDATE` changes 0 rows the second time

Known gap, say it before they do. If a *reply* is lost after being published, the recovery job re-sends the command, but the wallet answers from its stored result and does **not** emit a second reply, so the transfer can sit in `DEBITED`. The fix is small: when a command comes back as a replay, write the reply to the outbox again. This ties to the missing publisher confirms in section 13.

8.7 The in-between window

While a transfer is `DEBITED`, money has left A and not yet reached B. That is the price of a saga. The invariant to quote: **balances plus money in flight always equal the total money in the system**, and a retry never changes that total.

9. Resilience: surviving a bad dependency

9.1 The two disasters we defend against

- **Cascading failure.** Service A calls B. B slows down. A's threads pile up waiting, A starts timing out, and A's callers pile up too. One slow part drags the whole chain down.
- **Retry storm.** B has a brief outage. Thousands of clients all retry at the same instant when it returns, and the surge knocks it down again. The retries cause the outage to last longer.

9.2 The four tools (lecture 04, slide 7)

Tool	One-line meaning	Analogy	Our setting (`ProviderClient`)
Timeout	Never wait forever	Hanging up after a few rings	2 s per call, 1 s to connect
Retry with backoff and jitter	Try again, waiting longer each time, with a random wobble	Redialling a busy number after 1 min, then 2, then 4, each a bit random	Up to 4 attempts, 200 ms start, doubling, 50% randomness
Circuit breaker	After repeated failures, stop calling for a while and fail fast	A fuse that trips so the house does not burn	Opens at 50% failures over the last 6 calls (at least 4 calls)
Bulkhead	Give each dependency its own limited capacity	Watertight compartments in a ship	At most 10 concurrent provider calls

9.3 The circuit breaker's three states

State	Behaviour	Moves on when
CLOSED	Calls flow normally; failures are counted	Failure rate reaches 50%
OPEN	Calls are rejected instantly ("fail fast"), the provider is left alone to recover	After 10 seconds
HALF_OPEN	A couple of test calls are allowed through	Tests succeed: back to CLOSED. Tests fail: back to OPEN

You can watch it live at `GET /wallets/provider-status`.

9.4 How the tools are layered

```
Supplier<String> guarded = Retry.decorateSupplier(retry,
        CircuitBreaker.decorateSupplier(breaker,
                Bulkhead.decorateSupplier(bulkhead, call)));
```

Read it from the inside out: the call is limited by the bulkhead, watched by the breaker, and retried on failure. Retries are told **not** to retry when the breaker is open or the bulkhead is full, because those mean "stop", not "try again". When everything fails, the controller returns `503 PROVIDER_UNAVAILABLE` with a clear message instead of hanging.

9.5 Why retrying a payment is safe here

Retrying a charge is normally dangerous, since you might charge twice. It is safe because we send our own operation ID (`walletId:Idempotency-Key`) as the provider's `Idempotency-Key`, and the **mock provider remembers it**: the same key always returns the same `chargeId`. This is idempotency layer 3, and it is the reason resilience and idempotency belong in one project. **Retries are only safe on idempotent operations.**

9.6 The mock provider is deliberately nasty

`MockProviderController` records the charge **before** it may fail, to imitate the worst real case: "charged, but the answer never arrived". Two switches control it:

- `failPercent` (0 to 100): the share of calls that return HTTP 500.
- `delayMs`: adds latency, to trigger timeouts.

`GET /mock-provider/stats` reports `distinctCharges`. After a storm of retries it must equal the number of distinct keys, which proves no double charge.

9.7 Jitter elsewhere

`ConflictRetry` (ledger write collisions) also waits a random 5 to 25 ms times the attempt number. Same reasoning: competing requests must not retry in step.

9.8 What you should be able to demonstrate

Switch	You do	You see
`failPercent=60`	Send several card top-ups with new keys	Some succeed after retries, some end `503`; `distinctCharges` equals the number of keys
`failPercent=100`	Keep sending	`provider-status` becomes `OPEN`; later calls fail instantly
`failPercent=0`, wait 10 s	Send one call	`HALF_OPEN`, then `CLOSED` after success

10. Observability, chaos and testing

10.1 Observability (lecture 04, slide 8)

Definition. Observability is being able to understand what a running system is doing from the outside, using logs, metrics and traces. **Analogy:** the tracking number on a parcel. Without it you only know "it left" and "it never arrived".

Tool	What we have	Where
Trace ID	One ID per request. Accepted from the `X-Trace-Id` header (only letters, digits, `_` and `-`, up to 64 characters) or generated. It goes into the log lines, the response header and the `trace_id` column of every ledger event	`TraceFilter`, `LedgerService`
Structured log pattern	Every log line shows `[trace=...]`	`logging.pattern.level` in `application.yml`
Metrics	Counter `wallet.duplicates.suppressed`, tagged `layer=api` or `layer=inbox`	Micrometer through Actuator: `/actuator/metrics/wallet.duplicates.suppressed`

Tool	What we have	Where
Saga traceability	The saga passes `transfer-<transferId>` as the trace on the ledger events it causes, so a transfer can be followed from its ID	`CommandConsumer`

Honest limit: we have trace IDs, not a full distributed-tracing tool. OpenTelemetry with Jaeger would draw a timeline across services, and is listed as optional.

10.2 Chaos engineering

Definition. Break the system on purpose, in a controlled way, to prove it survives.
Analogy: a fire drill. You do not wait for a real fire to learn whether the exits work.

Switch	Where	What it simulates	What it proves
`dropResponse`	`POST /admin/chaos?dropResponse=true`	The server commits, then the answer is lost (HTTP 503)	A retry with the same key is a replay: money moved once
`duplicatePublish`	`.../chaos?duplicatePublish=true`	The relay publishes every message twice	The inbox skips the duplicate; the counter rises
`failConsumer`	`.../chaos?failConsumer=true`	A consumer that always throws	Retries, then messages land in the dead-letter queue
`failPercent`, `delayMs`	`/mock-provider/chaos`	A flaky or slow payment provider	Timeouts, retries with jitter, breaker, one charge per key

These endpoints are for demos only. A real system would never expose them, and the default RabbitMQ login (`guest`) is likewise for local use.

10.3 Testing and the *Component Testing* lecture

Definition. Component testing checks a piece in isolation; end-to-end tests check the pieces together. The lecture also covers stubs, drivers, mocks, boundary values, equivalence partitions, decision tables and state-transition testing.

Test	Kind	What it asserts
`RequestHasherTest`	Unit	Same input gives the same hash, different input a different one, length is 64
`IdempotencyE2ETest`: 50 parallel identical top-ups	End-to-end, concurrency	Exactly one ledger event; 49 responses are replays; balance 1000
`IdempotencyE2ETest`: same key, different amount	End-to-end	`422`, balance unchanged
`IdempotencyE2ETest`: withdrawal beyond balance	End-to-end	`409`
`IdempotencyE2ETest`: 8 parallel different keys	End-to-end, race	Balance 800 and `lastSeq` 8: no lost update
`TransferE2ETest`: success	End-to-end, saga	`COMPLETED`; balances 600 and 400; same key returns same transfer
`TransferE2ETest`: no funds	End-to-end, saga	`FAILED`, nothing moves
`TransferE2ETest`: missing destination	End-to-end, compensation	`COMPENSATED`, source balance restored
`k6/duplicates.js`	Load	50 virtual users, one key: final balance exactly 1000

The end-to-end tests are switched on with `-De2e=true` and need the services running; plain `mvn test` skips them.

10.4 Honest assessment: the weakest part for marks

The lecture asks for isolated component tests with stubs and mocks. Today the project has **one real unit test class** and a strong set of end-to-end tests. An examiner reading the *Component Testing* slides will notice. These additions are cheap and score well:

Add this test	Technique from the lecture
`IdempotentExecutor` with a mocked `JdbcTemplate`: the five outcomes in the table in 6.2	Decision table, mocks

Add this test	Technique from the lecture
`ConflictRetry`: succeeds on attempt 3; throws `TooMuchContentionException` after the maximum	Boundary values
`LedgerService.append`: amount equal to balance passes, balance minus one fails, zero amount rejected	Boundary value analysis, equivalence partitions
`SagaService.onReply`: every row of the transition table, plus a duplicate reply that must change nothing	State-transition testing
`ProviderClient` against a stubbed HTTP server: breaker opens after repeated 500s	Stubs
A property-style test: random mixed operations never break "sum of balances = top-ups minus withdrawals"	Invariant testing

Ask me for any of these one at a time and I will write it as a ready-to-paste JUnit class.

11. Java and Spring Boot elements

You already know Spring Boot basics, so this section ties each framework feature to the *reason* we use it. That reason is what a viva question usually asks.

11.1 The framework features

Element	Plain meaning	Why it is here	Where
Spring Boot starters and auto-configuration	Add a dependency and Boot configures it for you	Seven dependencies give us a web server, database access, migrations, a broker client and metrics	`pom.xml`
Dependency injection (IoC)	Spring builds objects and hands them what they need through the constructor	Services are small and replaceable, so tests can pass in fakes	every `@Service`, `@Component`
`@RestController`, `@RequestMapping`, `@PostMapping`	Turn methods into HTTP endpoints	The public REST API	`WalletController`, `TransferController`

Element	Plain meaning	Why it is here	Where
`@RequestHeader`	Reads an HTTP header	Reads `Idempotency-Key` and `X-Trace-Id`	controllers
Bean Validation (`@Valid`, `@Positive`, `@NotBlank`)	Declarative input rules	Rejects zero or negative amounts before any logic runs	request records
Java `record`	A compact immutable data holder	Request bodies and small internal results (`Stored`, `Row`)	controllers, executor
`@RestControllerAdvice`	One place that maps exceptions to HTTP responses	`422` for key reuse, `409` for no funds, `404` for a missing wallet	`ApiExceptionHandler`
`@Transactional`	Wrap a method in a database transaction	The heart of the project: key, effect and outbox commit together	`IdempotentExecutor`, `OutboxRelay`
Transaction propagation	How a method joins an existing transaction	`MANDATORY` makes `LedgerService.append` refuse to run outside one	`LedgerService`, `OutboxWriter`
`JdbcTemplate`	Plain SQL with minimal ceremony	We need `ON CONFLICT`, `FOR UPDATE SKIP LOCKED` and exact control over statements	most data code
`@Scheduled` and `@EnableScheduling`	Run a method on a timer	The outbox relay, the key cleanup, the saga recovery job	`OutboxRelay`, `IdempotencyCleanup`, `SagaService`
Spring AMQP (`RabbitTemplate`, `@RabbitListener`)	Send and receive RabbitMQ messages	Publishing outbox rows; consuming events, commands and replies	`messaging` packages
Listener retry settings	Retry a failing consumer, then reject	Feeds the dead-letter queue	`application.yml`
`OncePerRequestFilter`	Code that runs once around every request	Assigns the trace ID	`TraceFilter`
MDC (logging context)	Thread-local key/value pairs added to every log line	Puts `traceId` in logs	`TraceFilter`

Element	Plain meaning	Why it is here	Where
Actuator and Micrometer	Built-in health and metrics endpoints	The duplicates-suppressed counter	`/actuator/metrics/...`
Flyway	Runs versioned SQL at startup	Creates identical tables on every machine	`V1__init.sql`
Maven	Dependency and build tool	Two independent projects, one `pom.xml` each	project roots
Docker Compose	Describes several containers in one file	PostgreSQL and RabbitMQ with `docker compose up -d`	`docker-compose.yml`

11.2 Java language and library details worth knowing

- **Lambdas and `Supplier<T>`.** `IdempotentExecutor.execute(..., Supplier<Stored> effect)` takes "the work to do" as a parameter. The executor decides *whether and when* to run it.
- **`volatile` fields in `ChaosState`.** A change made by one thread (the admin call) must be visible to other threads (request handlers). `volatile` guarantees that visibility.
- **`ConcurrentHashMap.computeIfAbsent` in the mock provider.** An atomic "create if missing", so two simultaneous requests with one key get one `chargeId`.
- **`ThreadLocalRandom`** gives the jitter without sharing a lock between threads.
- **`java.net.http.HttpClient`** is the built-in HTTP client used by `ProviderClient` and by the tests, so no extra library is needed.
- **`jakarta.*` packages.** Modern Spring uses `jakarta.validation` and `jakarta.servlet`, not the old `javax.*`.

11.3 Why `JdbcTemplate` and not JPA/Hibernate

JPA hides SQL behind objects, which is productive for ordinary CRUD. Here the SQL **is** the design: `ON CONFLICT DO NOTHING`, `UNIQUE (wallet_id, seq)`, `FOR UPDATE SKIP LOCKED`, guarded `UPDATE ... WHERE state = ?`. With `JdbcTemplate` every statement is visible, explainable and testable. The cost is more hand-written SQL. If asked, say the trade-off was chosen deliberately.

11.4 Four Spring traps this project avoids (and one it accepts)

1. **`@Transactional` works through a proxy.** Calling a transactional method from another method in the *same class* skips the proxy and the transaction. That is why `IdempotentExecutor` is its own bean, called from controllers.
2. **A poisoned transaction cannot continue.** After a duplicate-key error the transaction must roll back. So retries are done *outside* the transactional method (`ConflictRetry`).
3. **Rolled-back-only marking.** Catching a business exception inside a transaction and carrying on can raise `UnexpectedRollbackException`. `CommandConsumer` therefore checks business rules with plain reads first.
4. **Rollback rules.** Spring rolls back on unchecked exceptions by default, not on checked ones. Our domain errors extend `RuntimeException` for that reason.
5. **Accepted shortcut.** Some JSON responses and the inter-service messages are built as strings (`key=value&key=value`). It avoids extra libraries and is easy to read, but a production system would use typed objects and a schema.

12. Project manager view

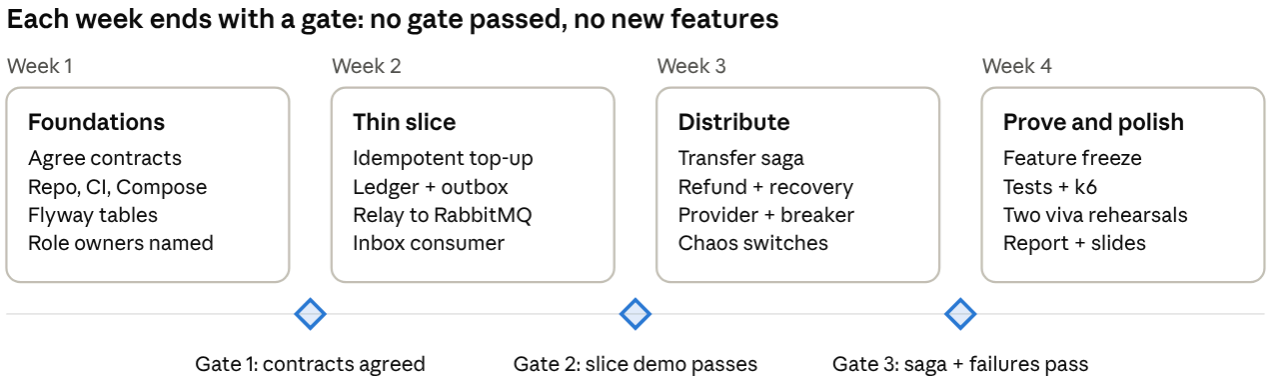
A project manager answers five questions: *what* are we building, *who* does what, *when*, *what could go wrong*, and *how do we know it is done*. This section answers each for your group. Dates are in weeks because I do not know your real submission date; map them once you do.

12.1 Charter in one table

Item	Statement
Objective	Show a working distributed wallet whose money-moving operations are safe under retries, duplicates and crashes
Success criteria	All 11 demo scenarios pass live; the 50-parallel-duplicate test shows exactly one effect; every member can explain every layer in the viva
In scope	Wallets, top-up, withdrawal, card top-up, transfer saga, outbox/inbox, DLQ, resilience, chaos switches, tests, report
Out of scope	Real bank or payment integration, real authentication, a polished UI, cloud deployment

Item	Statement
Stakeholders	The lecturer (client and assessor), the viva panel, the 8 group members
Constraints	8 people with uneven Java experience; a fixed deadline; limited AI-assistance usage; one stack (Java, Spring Boot)
Assumptions	Every laptop can run Docker; the code in the README compiles after small fixes; the viva may be individual

12.2 Roadmap



4-week roadmap - 4 phases, 3 gates

The four phases and three gates above are the plan. A **gate** is a go or no-go check: if the group does not pass it, the next week's work does not start. Final gate: the **feature freeze** at the start of week 4, after which only tests, fixes, the report and rehearsals are allowed. A smaller system that survives every demo beats a bigger one that only works on the happy path.

12.3 Work breakdown and ownership

#	Role	Deliverable	Done when
1	Idempotency lead	`idempotency/*`, `WalletController`, error handling	The 6.2 outcome table passes as tests
2	Ledger lead	`LedgerService`, `V1__init.sql`, replay endpoint	Overdraft race test passes; replay says `consistent`

#	Role	Deliverable	Done when
3	Saga lead	`SagaService`, `TransferController`, recovery job	Success, no-funds, compensation and crash scenarios pass
4	Messaging lead	RabbitMQ topology, relay, consumers, DLQ	Duplicate delivery is skipped; poison message reaches the DLQ
5	Resilience lead	`ProviderClient`, mock provider, card top-up	Breaker opens and closes; one charge per key
6	Chaos and load lead	Chaos switches, k6, the demo runbook	The demo runs end to end in under 10 minutes, twice
7	Quality lead	Unit and component tests, coverage, CI	The extra tests in 10.4 exist and pass
8	Observability, docs and slides lead	Trace ID, report, slides, evidence screenshots	Report matches the code; every claim has a screenshot

Each owner is also the **viva expert** for their part, but everyone must be able to explain the whole flow (section 13).

12.4 Dependencies and the critical path

The longest chain that cannot be shortened is: **agreed contracts → ledger → idempotent executor → outbox and relay → saga → chaos demo**. If the ledger slips, almost everything slips, so the ledger lead should be your strongest Java developer. Parallel work that does not block the chain: the mock provider, k6 scripts, the report skeleton, and test classes written against the agreed contracts.

12.5 Risk register

Risk	Likelihood	Impact	Mitigation	Owner
The README code has compile errors (it was never compiled by its author)	High	Medium	Fix on day one with one person; share the fixed commit; do not let eight people debug separately	1
Spring Boot 4 API differences surprise the team	Medium	Medium	Pin the version; search the error text with "Spring Boot 4 migration guide"	1

Risk	Likelihood	Impact	Mitigation	Owner
Uneven Java skill leaves one person carrying the work	High	High	Pair each weak member with a role owner; rotate who runs the demo	all
Individual viva exposes members who only built one part	High	High	Two rehearsals where each member explains a part they did not build	7
Docker or port problems on some laptops	Medium	Medium	A shared "setup check" script and a fallback machine for the demo	6
End-to-end tests are timing-sensitive and flake	Medium	Medium	Use the polling helper; never use fixed sleeps; run twice before demo day	7
Scope creep (UI, extra features)	Medium	High	Feature freeze gate; a "parking lot" list for ideas	PM
Demo fails live	Medium	High	Rehearse twice; keep a recorded backup of every scenario	6
Attribution issue with the reference repository	Low	High	State in the report that the idea was inspired by it and the code is a new Java design	8
AI-assistance limits reached near the deadline	Medium	High	Spend the budget early on the critical path; ask for small diffs, not whole files	PM

12.6 Definition of done (tick as you go)

- [ ] Both services start with `docker compose up -d` and two `mvn` runs, with no manual database edits
- [ ] All 11 demo scenarios pass on a clean machine
- [ ] `mvn test` passes; `mvn test -De2e=true` passes with both services running
- [ ] The extra component tests from 10.4 exist
- [ ] Report matches the code (names, ports, tables) and includes diagrams and screenshots
- [ ] Every member has explained every layer once, out loud, to another member
- [ ] The limitations in section 13 are written into the report

12.7 What examiners probably reward

I have not seen your assessment rubric, so treat this as an informed guess from the slides, and check it against the real brief.

Likely criterion	Evidence we show
Understanding of Idempotency and Exactly-Once(ish)	Three layers, the same-transaction rule, honest wording about "ish"
Use of distributed design patterns	Outbox, inbox, saga, CQRS and event sourcing, resilience
Working system and demo	11 live scenarios, failure first then the fix
Testing quality	Unit, component, end-to-end and load tests
Design justification	Why saga not 2PC, why optimistic concurrency, why JdbcTemplate
Teamwork and individual understanding	Roles, Git history, viva rehearsals

12.8 Ways of working

- **Cadence:** two 15-minute stand-ups a week (done, next, blocked), plus the weekly gate review.
- **Git:** one repository, protected `main`, a `feature/<role>` branch each, merge by pull request with one reviewer.
- **Decision log:** a short file listing each design decision, the options and the reason. It becomes your report's justification section almost for free.
- **Measures that matter:** demo scenarios passing (target 11 of 11), tests passing, and duplicates suppressed in the 50-parallel run (target 49).

13. Viva playbook, limitations, glossary and study plan

13.1 Strategy

1. **Lead with the problem, not the technology.** Panels remember the lost-reply story far better than a list of patterns.
2. **Always show failure first, then the fix.** Switch idempotency off, double-charge, then switch it on.
3. **Say "effectively-once", never "exactly-once".** Explain why the word "ish" is honest.
4. **Name your own limits before they do** (13.3). It reads as maturity, not weakness.

5. **Answer in your own words.** If you do not know, say how you would find out. Do not bluff.
6. **Point at code.** For each answer, open the one file that proves it.

A 60-second pitch you can adapt: "Our system is a digital wallet. The problem is that networks lose replies, so a customer's retry can move money twice. We solved it by giving every request an idempotency key and storing the key, the ledger event and the outgoing message in a single database transaction. Messages travel through an outbox to RabbitMQ and are deduplicated by an inbox on the other side. Transfers between wallets run as a saga with refunds. To prove it, we break the system on purpose: we drop responses, duplicate messages, crash a service and flood it with fifty identical requests, and the money always adds up."

13.2 Harder questions, beyond the ones in the earlier sections

Question	Strong answer
Isn't this just a unique constraint?	A unique constraint stops the duplicate but gives the client an error. We also store and replay the **original response**, check the **payload hash**, and make the effect and the record atomic.
What if the idempotency table is lost?	Then replays become new operations. That is why the ledger itself has `UNIQUE (wallet_id, seq)` and the saga commands have their own keys: defence in depth. In production the table lives in the same durable database as the ledger.
Who generates the key?	The **client**, once per intention (a UUID per "Send" tap), reused on every retry. A server-generated key would differ on each attempt and defeat the purpose.
What if a client uses a weak or guessable key?	Keys are scoped to endpoint and wallet, and a reused key with a different payload is rejected. In production, add the user ID to the scope.
What happens at 24 hours?	The row is deleted and a very late retry is treated as new. The TTL must exceed any realistic client retry window.

Question	Strong answer
Why RabbitMQ and not Kafka?	We need task-style queues, routing keys, acknowledgements and dead-letter queues, which RabbitMQ gives out of the box. Kafka suits replayable high-volume event logs; it would be heavier than needed here.
Why not store keys in Redis?	Redis is fast but is a **different system** from the ledger, so the key and the effect could not commit atomically. That would bring back the exact gap we closed.
Is outbox plus inbox overkill?	Each solves a different failure: the outbox stops a lost message on publish; the inbox stops a double effect on delivery. Remove either and a failure case reappears.
Why return 202 for a transfer?	The saga is asynchronous. 202 means "accepted, still in progress"; the client polls the transfer state.
How would it scale?	The ledger scales by partitioning wallets across instances (one writer per wallet), the relay by `SKIP LOCKED` with several instances, and consumers by competing consumers on the queue.
What if two relays publish the same row?	`SKIP LOCKED` prevents it in normal running. If it ever happened, the inbox dedupes it.
How do you know the total money is safe?	The invariant test: balances plus money in flight equal top-ups minus withdrawals, after every demo scenario.

13.3 Honest limitations (say these first)

Limitation	Effect	How to phrase it	Fix
No publisher confirms on the relay	A broker crash at the wrong moment could lose one message	"Delivery is at-least-once from the relay, but we do not yet use confirms."	Enable publisher confirms and mark `PUBLISHED` only on confirm

Limitation	Effect	How to phrase it	Fix
A replayed saga command emits no second reply	A lost reply could leave a transfer in `DEBITED`	"Recovery re-sends commands, but a stored result does not re-emit its reply."	On a replay, write the reply to the outbox again
A replayed card top-up still calls the provider first	A replay fails if the provider is down	"The provider is safe to call twice, but we could skip it."	Check the stored key before calling the provider
Balance projection is updated synchronously	It is always consistent, so it is simpler than true CQRS	"A deliberate simplification."	Update it asynchronously from the outbox stream
Keys are not tied to a user	No authentication exists	"Out of scope."	Add the user ID to the key scope
Messages are `key=value` strings	Fragile and untyped	"Chosen to avoid libraries."	JSON or Avro with a schema
Chaos endpoints are open and RabbitMQ uses default login	Insecure outside a demo	"Demo only."	Remove or protect them
No automatic replay of dead-letter messages	A human must act	"Operational follow-up."	A small admin job to inspect and re-publish
No snapshots	Replay slows on very long histories	"Not needed at this scale."	Snapshot every N events
Component tests are thin	Weaker marks on the testing lecture	Fix it, do not explain it	The additions in 10.4

13.4 Glossary

Term	Plain meaning
Idempotent	Doing it many times has the same effect as doing it once
Idempotency key	A client-chosen ID that names one intention, reused on every retry

Term	Plain meaning
Effectively-once	At-least-once delivery plus idempotent handling, so the effect happens once
Event sourcing	Store the history of facts, derive the state
CQRS	Separate model for writing and for reading
Projection	A read model built from events
Replay	Rebuilding state by re-applying events
Optimistic concurrency	Detect a clash when saving instead of locking first
Transaction	A group of changes that succeed or fail together
Outbox	A table of messages committed with the data, published later
Inbox	A table of processed message IDs used to skip duplicates
Relay	The worker that moves outbox rows to the broker
Dead-letter queue	A holding queue for messages that keep failing
Poison message	A message that always fails when processed
Saga	A chain of local transactions with compensations
Compensation	A new action that undoes the business effect of an earlier step
Orchestration	One coordinator directs the saga
Circuit breaker	Stops calls to a failing dependency for a while
Bulkhead	Limits capacity per dependency so one cannot starve the rest
Jitter	Random variation in retry delay
Retry storm	Many clients retrying together and overloading a recovering service
Trace ID	An identifier that follows one request through logs and events
Chaos engineering	Breaking things on purpose to prove resilience

13.5 Study plan

Five sessions of about 45 minutes. Each ends with a whiteboard drill, explaining out loud with no notes.

Session	Read	Whiteboard drill
1	Sections 1 and 2	Draw the lost-reply timeline and explain why a retry is unavoidable
2	Sections 4 and 5	Draw the overdraft race and how the unique constraint resolves it
3	Section 6	Draw the three layers and walk through 50 parallel identical requests
4	Sections 7 and 8	Draw outbox to relay to broker to inbox, then the saga state machine with a refund
5	Sections 9 to 13	Run the full demo once, then answer ten random questions from 13.2

When you can pass every drill without looking, you are ready. A practical test: explain one layer you did **not** build to a teammate who then tries to find a hole in it.